

Laboratorio 02: Heap File con Registros de Longitud Fija y Variable

Objetivo del Laboratorio

Implementar un **Heap File** para el manejo de registros de longitud fija y variable mediante un archivo binario organizado en **páginas de tamaño fijo**. Los estudiantes trabajarán con estrategias de organización en páginas, gestión del espacio libre y técnicas de eliminación (MOVE THE LAST, FREE LIST, SlottedPage), identificadores de registro (RID), fundamentales en sistemas de almacenamiento eficiente.

Estructura de Páginas

Todas las páginas deberán tener un tamaño fijo de:

PAGE_SIZE = 4096 bytes (para este laboratorio considerar un tamaño más pequeño)

El archivo deberá tener la siguiente estructura:

```
+-----+
|      FILE HEADER      |
+-----+
|      PAGE 0           |
+-----+
|      PAGE 1           |
+-----+
|      ...              |
+-----+
```

El **File Header** deberá contener como mínimo: page_size y num_pages. Opcionalmente, el estudiante podrá agregar información que facilite la localización de páginas con espacio disponible.

Cada registro deberá identificarse mediante: **RID = (page_id, slot_id)**

P1 (10 pts): Registros de Longitud Fija

Dada la siguiente estructura del registro de longitud fija:

Registro Alumno:

Atributos:

codigo (Cadena[5])	# Tamaño: 5 caracteres
nombre (Cadena[11])	# Tamaño: 11 caracteres
apellidos (Cadena[20])	# Tamaño: 20 caracteres
carrera (Cadena[15])	# Tamaño: 15 caracteres
ciclo (Entero)	# Entero para el ciclo
mensualidad (Decimal)	# Decimal para la mensualidad

Header de Página:

Para registros de longitud fija, el **header de cada página** debe contener:

- **Número total de registros** en la página
- **Número de registros activos** (no eliminados)
- **Puntero al primer espacio eliminado** (para FREE LIST strategy)

Requisitos:

Se le pide implementar una clase llamada `FixedRecord` que encapsule las operaciones de manipulación de archivo binario con dos estrategias de eliminación:

1. El **constructor** debe recibir el nombre del archivo y el modo de eliminación:

- **MOVE THE LAST**: mueve el último registro a la posición del registro eliminado.
- **FREE LIST**: mantiene una lista de espacios libres para ser usados en nuevas inserciones.
- Actualizar los metadatos de la página.
- Puede separar la implementación en dos clases, una para cada modo de eliminación.

2. **Implementar las siguientes funciones**:

- `load()` : devuelve todos los registros válidos del archivo.
- `add(record)` : agrega un nuevo registro utilizando la estrategia de localización de páginas propuesta.
- `readRecord(rid)` : obtiene el registro utilizando `(page_id, slot_id)` sin recorrer secuencialmente todo el archivo con $O(1)$.
- `remove(rid)` : elimina el registro utilizando la estrategia seleccionada con $O(1)$.

3. **Realizar pruebas funcionales** de cada método (`P1.py`).

- Generar **N registros** (mínimo 100) para las pruebas funcionales.
- Probar registros distribuidos en varias páginas.
- Probar inserción, lectura y eliminación.
- Verificar la reutilización de espacios con FREE LIST.
- Verificar el comportamiento de MOVE THE LAST.
- Verificar la creación de nuevas páginas cuando no exista espacio disponible.
- Mostrar el estado de los headers antes y después de las operaciones.

P2 (10 pts): Registros de Longitud Variable

Dada la siguiente estructura del registro de longitud variable:

Registro Matricula:

Atributos:

codigo (Cadena*)	# Tamaño variable
ciclo (Entero)	# Entero para el ciclo
mensualidad (Decimal)	# Decimal para la mensualidad
observaciones (Cadena*)	# Tamaño variable

Header de Página:

Para registros de longitud variable con **SlottedPage**, el **header de cada página** debe contener:

- **Número de slots** utilizados
- **Puntero al espacio libre** (free space pointer)
- **Directory de slots / ItemId Array**: array de pares (offset, length) para cada registro

Los registros deberán almacenarse desde el final de la página hacia el inicio, mientras que el Slot Directory crecerá desde el inicio hacia el final.

Requisitos:

Se le pide implementar un programa para leer y escribir registros de longitud variable en un archivo binario:

1. Usar **SlottedPage strategy** para manejar registros de longitud variable dentro de cada página.
2. Implementar las siguientes funciones:
 - `load()` : devuelve todos los registros del archivo.
 - `add(record)` : agrega un nuevo registro en una página que tenga espacio suficiente. Si ninguna página puede alojarlo, deberá crear una nueva página.
 - `readRecord(rid)` : obtiene el registro utilizando `(page_id, slot_id)` y la información almacenada en el Slot Directory con $O(1)$.
 - `remove(rid)` : proponer una estrategia de eliminación.
3. **Fragmentación**: El estudiante deberá identificar el problema de **fragmentación del espacio libre** producido por las eliminaciones. Como extensión opcional, se podrá implementar `compact()` para reorganizar los registros activos y consolidar el espacio libre.
4. Realizar **pruebas funcionales** de cada método (`P2.py`).
 - Generar **N registros** (mínimo 100).
 - Los registros deberán tener diferentes tamaños.
 - Probar inserción en varias páginas.
 - Probar lectura mediante RID.
 - Probar eliminación lógica.
 - Verificar correctamente el Slot Directory.
 - Verificar que `load()` no retorne registros eliminados.
 - Verificar la creación de nuevas páginas.
 - Mostrar el estado de una página antes y después de las operaciones.

Entregable

Archivos a entregar:

- [P1.py](#) y [P2.py](#) - Ejercicio 1 resuelto en Python (módulo struct)
- Archivos de datos utilizados durante las pruebas

- **Informe PDF:** Documentar brevemente la estructura del File Header y Page Header para cada tipo de registro, estrategia utilizada para localizar una pagina con espacio disponible, estrategia MOVE THE LAST y FREE LIST, estrategia de eliminación utilizada en P2, screenshots de ejecución satisfactoria, etc.

Criterios de evaluación:

Criterio	Puntos
Funcionamiento correcto de las operaciones	40%
Gestión de páginas, File Header y Page Header	25%
Implementación de las estrategias solicitadas	20%
Pruebas y casos de borde	10%
Calidad y documentación del código	5%
Total	100%